

Lección 14: Sentry Implementation

Setup Práctico sin Errores de CORS

Agenda

- ¿Qué es Sentry?
 - ¿Para Qué Sirve?
 - Configuración Paso a Paso
-

¿Qué es Sentry?

Definición:

Sentry es una plataforma de **monitoreo de errores y performance** en tiempo real para aplicaciones web y móviles.

En palabras simples:

- Captura errores que ocurren en producción
- Te notifica cuando algo se rompe
- Muestra **exactamente** qué pasó y por qué
- Graba sesiones de usuario para reproducir bugs

Como tener logs de consola de TODOS tus usuarios

¿Para Qué Sirve Sentry?

1. Error Tracking (seguimiento de errores):

// Usuario hace click → Error ocurre

✗ `TypeError: Cannot read property 'price' of undefined`

// Sin Sentry: No te enteras hasta que usuario se queja

// Con Sentry: Notificación instantánea + contexto completo

2. Performance Monitoring (monitoreo de rendimiento):

Checkout page: 250ms 

Checkout page: 4.5s  (regression detected!)

3. Session Replay (reproducción de sesiones):

- Graba lo que el usuario hizo antes del error
- Como tener un “video” de la sesión
- Sin violar privacidad (configurable)

¿Qué Problemas Resuelve? (1/2)

Problema 1: Errores Silenciosos

// Tu código en producción

```
try {  
  processPayment()  
} catch (e) {  
  console.error(e) // ✗ Nadie ve esto en producción  
}
```

Solución con Sentry:

```
try {  
  processPayment()  
} catch (e) {  
  Sentry.captureException(e) //  Te llega notificación  
}
```

¿Qué Problemas Resuelve? (2/2)

Problema 2: No Reproducibles

Usuario: "La app se rompió"

Dev: "¿Qué hiciste?"

Usuario: "No me acuerdo"

Dev: "No puedo reproducirlo" 🤖

Solución con Sentry:

Sentry muestra:

- ✓ Qué pasos hizo el usuario (breadcrumbs)
- ✓ En qué navegador/OS
- ✓ Qué datos tenía en memoria
- ✓ Replay de la sesión (video)
- ✓ Stack trace completo

Resultado: Bug reproducible en 5 minutos

Casos de Uso Reales

Startup E-commerce:

Sin Sentry:

- 50 usuarios reportan "no puedo pagar"
- 2 días debugging sin reproducir
- \$10,000 en ventas perdidas

Con Sentry:

- Error detectado en 30 segundos
 - Stack trace muestra API de pagos caída
 - Fix en 20 minutos
 - \$200 de pérdida (vs \$10,000)
-

Casos de Uso: Performance

SaaS Application:

Sentry detectó:

📊 Dashboard load: 250ms → 8s (regression!)

🔍 Análisis: Query N+1 en nueva feature

➤ Fix: Agregamos caching

📈 Resultado: 250ms restored

Sin Sentry: Usuarios se quejan después de 1 semana

Con Sentry: Detectado en 1 hora, fix antes de quejas

¿Cuándo Usar Sentry?

✅ Úsalo si:

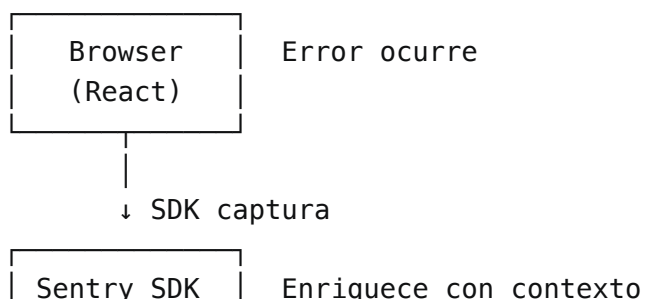
- App en producción con usuarios reales
- Necesitas saber cuándo algo falla
- Quieres mejorar performance
- Debugging es difícil (muchos usuarios, diferentes navegadores)

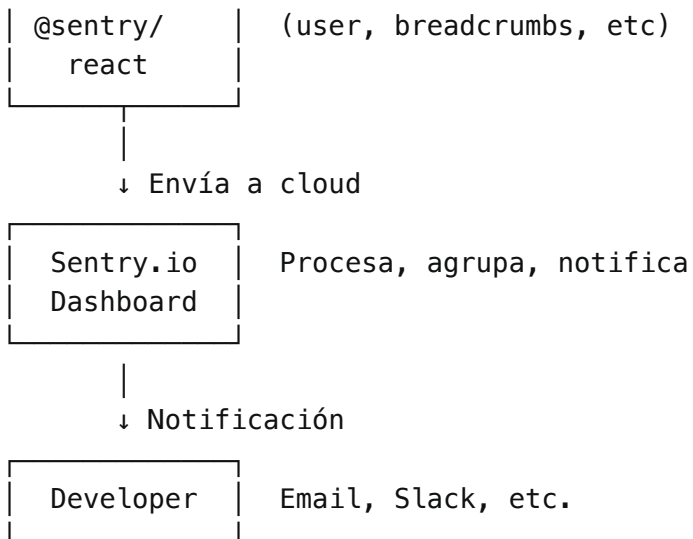
❌ No lo necesitas si:

- Proyecto personal local
- Prototipo que no llega a producción
- Ya tienes solución enterprise (Datadog, New Relic)

Para este curso: Lo usamos para aprender error tracking profesional

Arquitectura de Sentry





Alternativas a Sentry

Otros servicios similares:

- **Rollbar**: Más simple, menos features
- **Bugsnag**: Similar a Sentry
- **LogRocket**: Enfocado en session replay
- **Datadog**: Enterprise, más caro
- **New Relic**: Performance monitoring

Por qué Sentry:

- ☒ Free tier generoso (5,000 errors/mes)
- ☒ Open source (self-hosted opcional)
- ☒ Documentación excelente
- ☒ Integraciones con todo
- ☒ Estándar de la industria

Instalación de Sentry (1/2)

Paso 1: Instalar paquete:

```
pnpm add @sentry/react
```

Paso 2: Crear cuenta en [Sentry.io](https://sentry.io):

1. Ir a <https://sentry.io/signup/> y crear cuenta gratis
2. En el dashboard: **Settings** → **Projects** → **Create Project**
3. Seleccionar plataforma: **React**
4. Nombre del proyecto: “ai-course” (o el que prefieras)
5. Copiar el **DSN** (Data Source Name) que aparece

Instalación de Sentry (2/2)

DSN Format:

```
https://[PUBLIC_KEY]@[ORG_ID].ingest.[REGION].sentry.io/[PROJECT_ID]
```

Ejemplo:

```
https://643641154a374449476e22afaf003508@o4510005722152960.ingest.de.sentry
```

Paso 2a: Extraer Valores del DSN (1/2)

Tu DSN de Sentry:

```
https://examplePublicKey123@o4508888888888888.ingest.us.sentry.io/4508999999
```

PUBLIC_KEY

ORG_ID

REGION

PROJECT

Componentes:

- **PUBLIC_KEY:** examplePublicKey123 (parte pública del DSN)
- **ORG_ID:** 4508888888888888 (ID de tu organización)
- **REGION:** us (puede ser us , eu , de , etc.)
- **PROJECT_ID:** 4508999999999999 (ID de tu proyecto específico)

Paso 2b: Extraer Valores del DSN (2/3)

Dónde encontrar tu DSN en [Sentry.io](https://sentry.io):

1. Crear proyecto: **Settings** → **Projects** → **Create Project** → **React**
2. En la barra lateral (sidebar): **SDK Setup** → **Client Keys (DSN)**
3. Copiar el DSN completo que aparece
4. **Ejemplo de DSN copiado:**

```
https://abc123def456@o1234567890.ingest.us.sentry.io/9876543210
```

Paso 2b: Extraer Valores del DSN (3/3)

Ahora extrae los valores:

DSN: `https://abc123def456@o1234567890.ingest.us.sentry.io/9876543210`

ORG_ID:	1234567890	(números después de 'o')
REGION:	us	(antes de .sentry.io)
PROJECT_ID:	9876543210	(números al final)

Usar estos valores en:

- `src/infrastructure/sentry.ts` → DSN completo
- `vite.config.ts` → ORG_ID, REGION, PROJECT_ID para tunnel

Propiedades de Configuración Principales

1. DSN (Data Source Name):

```
dsn: 'https://...' // Identifica tu proyecto en Sentry
```

2. Environment (entorno):

```
environment: import.meta.env.DEV ? 'development' : 'production'  
// Separa errores de dev vs prod en dashboard
```

3. Release (versión):

```
release: 'app@1.0.0'  
// Trackea en qué versión ocurre el error
```

4. Debug:

```
debug: true // Logs detallados en consola (solo desarrollo)
```

Sample Rates

Performance Monitoring:

```
tracesSampleRate: 1.0 // 100% en dev, 0.1 (10%) en prod  
// Captura transacciones de performance
```

Profiling:

```
profilesSampleRate: 1.0 // 100% en dev, 0.1 en prod  
// Perfiles de performance del código
```

Session Replay:

```
replaysSessionSampleRate: 1.0 // 100% en dev, 0.1 en prod  
replaysOnErrorSampleRate: 1.0 // 100% cuando hay error  
// Graba sesiones de usuario
```

Por qué 1.0 en dev, 0.1 en prod: Costos de Sentry y volumen de datos

Integraciones Disponibles

1. Browser Tracing Integration:

```
Sentry.browserTracingIntegration({
  enableLongTask: true, // Detecta tareas que bloquean UI
  enableInp: true, // Interaction to Next Paint
})
```

Qué captura: Page loads, navegación, interacciones

2. Replay Integration:

```
Sentry.replayIntegration({
  maskAllText: false, // Ocultar texto sensible
  maskAllInputs: false, // Ocultar inputs
  blockAllMedia: false, // Bloquear videos/imágenes
})
```

Qué graba: Clicks, scrolls, inputs, navegación (como video)

Integraciones Avanzadas

3. Feedback Integration:

```
Sentry.feedbackIntegration({
  colorScheme: 'system',
  formTitle: 'Reportar Problema',
  submitButtonLabel: 'Enviar',
})
```

Qué hace: Widget para que usuarios reporten bugs

4. Auto Session Tracking:

```
autoSessionTracking: true
```

Qué trackea: Sesiones de usuario, crashes, stability score

5. Attach Stack Trace:

```
attachStacktrace: true
```

Qué envía: Stack trace completo en todos los eventos

Context

User Context:

```
Sentry.setUser({  
  id: 'user-123',  
  email: 'user@example.com',  
  username: 'john_doe',  
})
```

Por qué: Identificar qué usuarios afecta el error

Business Context:

```
Sentry.setContext('business', {  
  plan: 'premium',  
  subscription_status: 'active',  
})
```

Por qué: Filtrar errores por tipo de cliente

Tags

Tags para Filtros:

```
Sentry.setTag('feature', 'checkout')  
Sentry.setTag('experiment', 'new-ui')
```

Por qué: Buscar errores por feature en dashboard

Breadcrumbs (migas de pan) (1/2)

Qué son: Historial de eventos antes del error

Automáticos (Sentry captura solo):

- Clicks de usuario
- Cambios de URL
- Console logs
- Fetch/XHR requests

Breadcrumbs (migas de pan) (2/2)

Manuales (tú agregas):

```
Sentry.addBreadcrumb({
  message: 'User added item to cart',
  category: 'user.action',
  level: 'info',
  data: {
    product_id: '123',
    quantity: 2,
  },
})
```

Por qué son útiles: Ves qué hizo el usuario antes del crash

Tracking de Negocio

```
// En CartSummary.tsx
const handleClearCart = () => {
  Sentry.addBreadcrumb({
    message: 'User cleared cart',
    data: {
      items_count: itemCount,
      subtotal: subtotal,
      total_discounts: totalDiscounts,
    },
  },
```

```
    level: 'info',
  })

  clearCart()
}
```

Ejercicio 1: Inicializar Sentry con Tunnel Proxy

Prompt:

Actúa como un SRE (Site Reliability Engineer) implementando error monitorin

CONTEXT0: Sentry = plataforma de monitoreo de errores y performance en tiem real. Captura errores en producción + notifica + muestra contexto completo (como tener logs de consola de TODOS los usuarios). Problema localhost: ad-blockers y firewalls bloquean requests directos a sentry.io → NetworkErr Solución: Tunnel proxy redirige requests de Sentry a través de tu servidor [local](#) (/tunnel) evitando bloqueadores. DSN (Data Source Name): URL única qu identifica tu proyecto Sentry. Environment: development vs production (sepa errores por entorno). Debug mode: muestra logs de Sentry en consola (útil p verificar setup).

TAREA: Inicializa Sentry en main.tsx con tunnel proxy para development.

INSTALACIÓN:

- Comando: `pnpm add @sentry/react`
- SDK: React-specific con React error boundary integration

OBTENER DSN:

1. Crear cuenta en [sentry.io](#) (gratis)
2. Crear nuevo proyecto: Platform = React
3. Copiar DSN: `https://xxx@xxx.ingest.sentry.io/xxx`

CONFIGURACIÓN SENTRY.INIT:

- Archivo: `src/main.tsx`
- Import: `import * as Sentry from '@sentry/react'`
- Ubicación: ANTES de `ReactDOM.render()` (inicializar primero)

OPTIONS REQUERIDAS:

- `dsn`: [tu DSN de [sentry.io](#)]
- `tunnel`: `import.meta.env.DEV ? '/tunnel' : undefined` (proxy solo en dev)
- `environment`: `import.meta.env.DEV ? 'development' : 'production'`
- `debug`: `true` (mostrar logs en consola para verificar)
- `integrations`: [

```
Sentry.browserTracingIntegration(), (performance)
Sentry.replayIntegration(), (session replay)
]
```

VALIDACIÓN:

- Ejecutar: `pnpm run dev`
- Consola debe mostrar:
 - `"[Sentry] Sentry initialized"`
 - `"[Sentry] Using tunnel: /tunnel"`
- Si NO aparecen logs → revisar debug: `true`

Aprende: Tunnel proxy soluciona CORS/ad-blocker

issues en desarrollo local

Ejercicio 2: Capturar Error Manual con User Context

Prompt:

Actúa como un developer testeando Sentry error capturing con user context.

CONTEXT0: `Sentry.captureException()` envía errores manualmente a Sentry (útil para try/catch custom). User context (`Sentry.setUser()`) adjunta info de usuario al error: facilita reproducir bugs (saber QUÉ usuario experimentó el error) Breadcrumbs: trail de eventos (clicks, navigations) antes del error (contexto de QUÉ hizo usuario). Sentry dashboard Issues: lista de errores agrupados por tipo. Manual capture vs auto capture: manual = try/catch controlado, auto = errores no manejados. Testing error capture: crear error intencional para verificar setup funciona.

TAREA: Crea botón que captura error manual en Sentry con user context.

COMPONENTE:

- Archivo: `src/components/TestErrorButton.tsx`
- Propósito: Botón de testing para verificar Sentry funciona
- Uso: Solo en development (NO incluir en producción)

IMPLEMENTACIÓN:

- Import: `import * as Sentry from '@sentry/react'`
- Component: `TestErrorButton` functional component
- Handler: `handleClick` **function**

LÓGICA handleClick:

1. Set user context: `Sentry.setUser({ id: 'test-user', email: 'test@example`
2. Capture exception: `Sentry.captureException(new Error('Test error from bu`
3. Log confirmación: `console.log('Error sent to Sentry')`

UI:

- Botón simple: `<button onClick={handleClick}>Test Error</button>`
- Styling opcional: Tailwind classes para visibilidad

INTEGRACIÓN:

- Importar `TestErrorButton` en `App.tsx` o `Dashboard`
- Renderizar condicionalmente: `{import.meta.env.DEV && <TestErrorButton />}`

VALIDACIÓN EN SENTRY DASHBOARD:

1. Click botón **"Test Error"**
2. Ir a `sentry.io` → Projects → [tu proyecto] → Issues
3. Verificar nuevo issue:
 - Title: **"Test error from button"**
 - User: test-user (`test@example.com`)
 - Environment: development
 - Breadcrumbs: debe mostrar click event antes del error
4. Tiempo de aparición: ~5-10 segundos después del click

VALIDACIÓN: Error debe aparecer en Sentry dashboard con user context comple

Aprende: Manual error capture + user context facilita

debugging de issues específicos

Puntos Clave

1. **Instalación Simple:** `pnpm add @sentry/react` + crear proyecto en sentry.io
2. **DSN:** Identifica tu proyecto, obtenerlo del dashboard
3. **Tunnel + Proxy:** Solución definitiva para evitar CORS y ad-blockers
4. **Sample Rates:** 1.0 (100%) en dev, 0.1 (10%) en prod para costos
5. **Integraciones:** browserTracing (performance), replay (sesiones), feedback (reportes)
6. **Context:** `setUser`, `setContext`, `setTag` para metadata rica
7. **Breadcrumbs:** Historial automático + manual de eventos antes del error
8. **beforeSend:** Hook para filtrar o modificar eventos antes de enviar
9. **Sin compromisos:** Testing real en desarrollo sin CORS errors

10. **Production-ready:** Misma configuración funciona en prod